

实验报告

课程:机器学习	实验名称: k 近邻算法	日期:3.30
姓名:雷文豪	学号:138022024031	专业班级:人工智能

1 实验目的

1. 掌握 k 近邻分类算法、 k 近邻回归算法的实现，学会在代码中数据的存储和使用。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 给待解问题设计求解方法和适合的模型。
2. 对给定数据集，设计适合模型的求解方法、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 昆虫分类问题（独立求解）

对如下 2 种昆虫的数据集（类别，触长、翅长），请设计合适的 k 近邻模型和方法，预测触长 1.2、翅长 1.8 昆虫的类别。

昆虫数据集中 A 昆虫有 6 个样本：

触长	1.1	1.1	1.2	1.2	1.3	1.3
翅长	1.8	1.9	1.9	2.0	2.2	2.3

表 1: 昆虫 A 的样本

昆虫数据集中 B 昆虫有 6 个样本：

触长	1.3	1.3	1.4	1.4	1.5	1.5
翅长	1.6	1.5	1.6	1.7	1.6	1.7

表 2: 昆虫 B 的样本

1. 简单写出你的求解设计（例如： k 近邻分类模型包括哪些组成和参数、如何确定组成和参数、如何预测新样本的类别）。
2. 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。
3. 获得的结果，即分类模型及其预测结果。
4. 程序运行成功的截图（包含源文件名<字母+学号后3位>、运行光标在程序尾行）。

1.求解设计

手动实现一个 k 近邻分类器，关键组件与参数如下：

- 距离度量：欧氏距离 $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- 分类决策规则：多数投票法（取 k 个最近邻中出现次数最多的类别）
- k 值选取：分别测试 $k=1, 3, 5$

预测流程如下：

1. 计算待预测样本与12个训练样本的欧氏距离
2. 按距离从小到大排序
3. 取前 k 个最近邻
4. 统计 k 个近邻中A类和B类的数量
5. 返回票数最多的类别作为预测结果

2.源代码

```
import math
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.sans serif'] = ['SimHei', 'Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False
```

===== 数据准备 =====

A类鸢尾花

```
A_length = [1.1, 1.1, 1.2, 1.2, 1.3, 1.3]
```

```
A_width = [1.8, 1.9, 1.9, 2.0, 2.2, 2.3]
```

B类鸢尾花

```
B_length = [1.3, 1.3, 1.4, 1.4, 1.5, 1.5]
```

```
B_width = [1.6, 1.5, 1.6, 1.7, 1.6, 1.7]
```

构造特征矩阵和标签

```
X = []
```

```
y = []
```

```
for i in range(6):
```

```
    X.append([A_length[i], A_width[i]])
```

```
    y.append('A')
```

```
for i in range(6):
```

```
    X.append([B_length[i], B_width[i]])
```

```
    y.append('B')
```

```
test_sample = [1.2, 1.8] 待预测样本
```

```
===== k NN 算法实现 =====
```

```
def euclidean_distance(a, b):
```

```
    """计算两个点之间的欧氏距离"""
```

```
    return math.sqrt((a[0] - b[0])2 + (a[1] - b[1])2)
```

```
def knn_classify(X_train, y_train, x_test, k):
```

```
    """
```

```
    k 近邻分类器
```

步骤:

1. 计算测试样本与所有训练样本的距离
2. 按距离升序排序，取前k个最近邻
3. 统计k个近邻中各标签的出现次数
4. 返回出现次数最多的标签作为预测结果

```
    """
```

计算所有训练样本到测试样本的距离

```
distances = []
```

```
for i, x_train in enumerate(X_train):
```

```
    d = euclidean_distance(x_train, x_test)
```

```
    distances.append((d, y_train[i]))
```

按距离排序

```
distances.sort(key=lambda item: item[0])
```

取前k个的类别进行投票

```
k_neighbors = distances[:k]
```

```
votes = {}
```

```
for _, label in k_neighbors:
```

```
    votes[label] = votes.get(label, 0) + 1
```

返回票数最多的类别

```
return max(votes, key=votes.get)
```

===== 分类预测 =====

```
print("=" 60)
```

```
print("任务3.1: 鸢尾花分类 — k NN分类器")
```

```
print("=" 60)
```

```
print(f"待预测样本: 萼片长度={test_sample[0]}, 萼片宽度={test_sample[1]}")
```

```
print()
```

```
for k in [1, 3, 5]:
```

```
    pred = knn_classify(X, y, test_sample, k)
```

```
    print(f"k = {k} 时, 预测类别: {pred}")
```

详细输出各距离

```
print("\n" + " " 40)
```

```
print("各训练样本到测试样本的距离: ")
```

```
for i, (x_i, label) in enumerate(zip(X, y)):
```

```
    d = euclidean_distance(x_i, test_sample)
```

```
    print(f" 样本{i+1}: 特征={x_i}, 类别={label}, 距离={d:.4f}")
```

排序后的近邻

```
distances = [(euclidean_distance(x_i, test_sample), label, x_i) for x_i, label in zip(X, y)]
```

```
distances.sort(key=lambda item: item[0])
```

```
print("\n按距离排序后的近邻序列: ")
```

```
for rank, (d, label, feat) in enumerate(distances):
```

```
    print(f" 第{rank+1}近邻: 距离={d:.4f}, 类别={label}, 特征={feat}")
```

===== 可视化 =====

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

左图：样本分布

```
ax = axes[0]
a_pts = ax.scatter(A_length, A_width, c='blue', marker='o', s=80, label='A类鸢尾花', zorder=5)
b_pts = ax.scatter(B_length, B_width, c='red', marker='s', s=80, label='B类鸢尾花', zorder=5)
test_pt = ax.scatter(test_sample[0], test_sample[1], c='green', marker=' ', s=200,
                     edgecolors='black', linewidths=1.5, label='待预测样本 (1.2, 1.8)', zorder=10)
ax.set_xlabel('萼片长度')
ax.set_ylabel('萼片宽度')
ax.set_title('鸢尾花样本分布与待预测样本')
ax.legend()
ax.grid(True, alpha=0.3)
ax.set_xlim(0.9, 1.7)
ax.set_ylim(1.3, 2.5)
```

右图：k=1,3,5 的决策可视化

```
ax = axes[1]
重新绘制所有样本点
ax.scatter(A_length, A_width, c='blue', marker='o', s=80, label='A类', zorder=5)
ax.scatter(B_length, B_width, c='red', marker='s', s=80, label='B类', zorder=5)
ax.scatter(test_sample[0], test_sample[1], c='green', marker=' ', s=200,
          edgecolors='black', linewidths=1.5, zorder=10)

标注每个样本点
for i, (lx, ly, lbl) in enumerate(zip(A_length + B_length, A_width + B_width, y)):
    ax.annotate(f'{i+1}', (lx, ly), textcoords="offset points", xytext=(5, 5), fontsize=8)

绘制到测试点连线（所有近邻）
distances_sorted = sorted([(euclidean_distance(x_i, test_sample), x_i) for x_i in X],
                          key=lambda item: item[0])
for k_val, color in [(1, 'orange'), (3, 'purple'), (5, 'cyan')]:
    for j in range(k_val):
        d, feat = distances_sorted[j]
        ax.plot([test_sample[0], feat[0]], [test_sample[1], feat[1]],
                color=color, linestyle=' ', linewidth=1, alpha=0.6)
```

图例

```
from matplotlib.lines import Line2D
custom_lines = [
    Line2D([0], [0], color='orange', linestyle=' ', label='k=1 近邻'),
    Line2D([0], [0], color='purple', linestyle=' ', label='k=3 近邻'),
    Line2D([0], [0], color='cyan', linestyle=' ', label='k=5 近邻'),
]
leg2 = ax.legend(handles=custom_lines, loc='lower right', fontsize=8)
ax.add_artist(ax.legend(loc='upper right'))
ax.legend(loc='upper right', fontsize=8)
ax.add_artist(leg2)

ax.set_xlabel('萼片长度')
ax.set_ylabel('萼片宽度')
ax.set_title('不同k值的近邻选择')
ax.grid(True, alpha=0.3)
ax.set_xlim(0.9, 1.7)
ax.set_ylim(1.3, 2.5)

plt.tight_layout()
plt.savefig('task3_1_iris_classification.png', dpi=150, bbox_inches='tight')
plt.close()
print("\n可视化结果已保存为 task3_1_iris_classification.png")
```

3.获得的结果

=====

任务3.1：鸢尾花分类 — k NN分类器

=====

待预测样本: 萼片长度=1.2, 萼片宽度=1.8

k = 1 时, 预测类别: A

k = 3 时, 预测类别: A

k = 5 时, 预测类别: A

各训练样本到测试样本的距离:

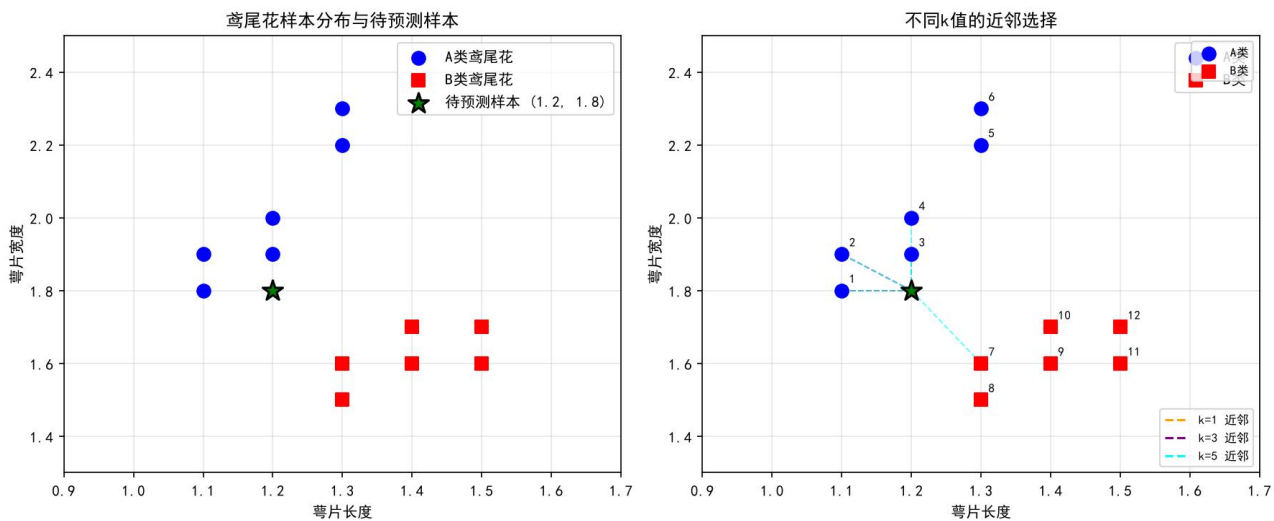
样本1: 特征=[1.1, 1.8], 类别=A, 距离=0.1000

样本2: 特征=[1.1, 1.9], 类别=A, 距离=0.1414
样本3: 特征=[1.2, 1.9], 类别=A, 距离=0.1000
样本4: 特征=[1.2, 2.0], 类别=A, 距离=0.2000
样本5: 特征=[1.3, 2.2], 类别=A, 距离=0.4123
样本6: 特征=[1.3, 2.3], 类别=A, 距离=0.5099
样本7: 特征=[1.3, 1.6], 类别=B, 距离=0.2236
样本8: 特征=[1.3, 1.5], 类别=B, 距离=0.3162
样本9: 特征=[1.4, 1.6], 类别=B, 距离=0.2828
样本10: 特征=[1.4, 1.7], 类别=B, 距离=0.2236
样本11: 特征=[1.5, 1.6], 类别=B, 距离=0.3606
样本12: 特征=[1.5, 1.7], 类别=B, 距离=0.3162

按距离排序后的近邻序列:

第1近邻: 距离=0.1000, 类别=A, 特征=[1.1, 1.8]
第2近邻: 距离=0.1000, 类别=A, 特征=[1.2, 1.9]
第3近邻: 距离=0.1414, 类别=A, 特征=[1.1, 1.9]
第4近邻: 距离=0.2000, 类别=A, 特征=[1.2, 2.0]
第5近邻: 距离=0.2236, 类别=B, 特征=[1.3, 1.6]
第6近邻: 距离=0.2236, 类别=B, 特征=[1.4, 1.7]
第7近邻: 距离=0.2828, 类别=B, 特征=[1.4, 1.6]
第8近邻: 距离=0.3162, 类别=B, 特征=[1.3, 1.5]
第9近邻: 距离=0.3162, 类别=B, 特征=[1.5, 1.7]
第10近邻: 距离=0.3606, 类别=B, 特征=[1.5, 1.6]
第11近邻: 距离=0.4123, 类别=A, 特征=[1.3, 2.2]
第12近邻: 距离=0.5099, 类别=A, 特征=[1.3, 2.3]

可视化结果已保存为 task3_1_iris_classification.png



4.运行截图

```

137 line2([0], [0], color="orange", linestyle="--", label="k=1 近邻"),
138 line2([0], [0], color="purple", linestyle="--", label="k=3 近邻"),
139 line2([0], [0], color="cyan", linestyle="--", label="k=5 近邻"),
140 ]
141 leg2 = ax.legend(handles=custom_lines, loc="lower right", fontsize=8)
142 ax.add_artist(ax.legend(loc="upper right"))
143 ax.legend(loc="upper right", fontsize=8)
144 ax.add_artist(leg2)
145
146 ax.set_xlabel("萼片长度")
147 ax.set_ylabel("萼片宽度")
148 ax.set_title("不同k值的近邻选择")
149 ax.grid(True, alpha=0.3)
150 ax.set_xlim(0.9, 1.7)
151 ax.set_ylim(1.3, 2.5)
152
153 plt.tight_layout()
154 plt.savefig('task3_1_iris_classification.png', dpi=150, bbox_inches='tight')
155 plt.close()
156 print("可视化结果已保存为 task3_1_iris_classification.png")
157

```

(machine_learning) PS D:\Studies\机器学习\src\exp4 & D:\Winconda\envs\machine_learning\python.exe d:/Studies/机器学习/src/exp4/task3_1_iris_classification.py
 k = 1 时, 预测类别: A
 k = 3 时, 预测类别: A
 k = 5 时, 预测类别: A

输出样本到测试样本的距离:
 样本1: 特征-[1.3, 1.8], 类别-A, 距离-0.1089
 样本2: 特征-[1.3, 1.9], 类别-A, 距离-0.1414
 样本3: 特征-[1.2, 1.9], 类别-A, 距离-0.1089
 样本4: 特征-[1.2, 2.0], 类别-A, 距离-0.2089
 样本5: 特征-[1.3, 2.2], 类别-A, 距离-0.4123
 样本6: 特征-[1.3, 2.3], 类别-A, 距离-0.5089
 样本7: 特征-[1.3, 1.6], 类别-A, 距离-0.2236
 样本8: 特征-[1.5, 1.5], 类别-B, 距离-0.3162
 样本9: 特征-[1.4, 1.6], 类别-B, 距离-0.2038
 样本10: 特征-[1.4, 1.7], 类别-B, 距离-0.2236
 样本11: 特征-[1.5, 1.6], 类别-B, 距离-0.3080
 样本12: 特征-[1.5, 1.7], 类别-B, 距离-0.3162

按距离排序后的近邻序列:
 第1近邻: 距离-0.1089, 类别-A, 特征-[1.3, 1.8]
 第2近邻: 距离-0.1089, 类别-A, 特征-[1.2, 1.9]
 第3近邻: 距离-0.1414, 类别-A, 特征-[1.3, 1.9]
 第4近邻: 距离-0.2089, 类别-A, 特征-[1.2, 2.0]
 第5近邻: 距离-0.2236, 类别-A, 特征-[1.3, 1.6]
 第6近邻: 距离-0.2236, 类别-A, 特征-[1.4, 1.7]
 第7近邻: 距离-0.2038, 类别-B, 特征-[1.4, 1.6]
 第8近邻: 距离-0.3162, 类别-B, 特征-[1.3, 2.3]
 第9近邻: 距离-0.3162, 类别-B, 特征-[1.5, 1.7]
 第10近邻: 距离-0.3080, 类别-B, 特征-[1.5, 1.6]
 第11近邻: 距离-0.4123, 类别-A, 特征-[1.3, 2.2]
 第12近邻: 距离-0.5089, 类别-A, 特征-[1.3, 2.3]

可视化结果已保存为 task3_1_iris_classification.png
 (machine_learning) PS D:\Studies\机器学习\src\exp4 &

3.2 身体指标.体脂率关系问题（模仿实例例题求解）

对体脂（bodyfat.txt）数据集¹，请设计合适的 k 近邻模型分析身体指标与体脂率之间的关系，并评估所建立模型的回归效果。

1. 简单写出你的求解设计（例如：采用哪种回归模型、模型的具体形式、哪个属性被设置为回归目标、如何求出目标变量的值、如何评估回归效果）。
2. 获得的结果，即写出具体的模型和回归效果。
3. 程序运行成功的截图（包含源文件名<字母+学号后3位>、运行光标在程序尾行）。

1.求解设计

使用 bodyfat.txt 数据集（252个样本），建立 k NN 回归模型分析身体各项指标与体脂率之间的关系，并评估回归效果。

1.1数据集说明

- 数据来源: [Kaggle Body Fat Prediction Dataset](https://www.kaggle.com/datasets/fedesoriano/body-fat-prediction-dataset)
- 特征 (13维): Age, Weight, Height, Neck, Chest, Abdomen, Hip, Thigh, Knee, Ankle, Biceps, Forearm, Wrist
- 目标变量: BodyFat (体脂率, %), 范围 0.00% ~ 47.50%

1.2模型设计

手动实现一个k近邻回归器, 关键设计选择如下:

- 数据预处理: Z score标准化, 消除量纲影响, 标准化公式: $x' = (x - \text{mean}) / \text{std}$
- 距离度量: 欧氏距离 (13维空间), $d = \sqrt{\sum (x_i - y_i)^2}$
- 回归预测: 对k个最近邻的目标值取算术平均, $\text{pred} = (1/k) \sum y_{\text{neighbor}}$
- 评估方法: 留一法交叉验证 (LOOCV), 每次留1个样本测试, 其余251个训练, 共252轮, 评估指标为MSE, RMSE, MAE, R^2
- k值选择: 测试 k=1, 3, 5, 7, 10

2.获得的结果

任务3.2: 身体指标与体脂率 — k NN回归器

数据集样本数: 252

特征维度: 13

目标变量: BodyFat (体脂率%)

体脂率范围: 0.00% ~ 47.50%

留一法交叉验证 (Leave One Out CV) 评估结果:

k= 1 MSE=42.7402 RMSE=6.5376 MAE=5.5087 R2=0.3873

k= 3 MSE=29.7704 RMSE=5.4562 MAE=4.5552 R2=0.5732

k= 5 MSE=28.4355 RMSE=5.3325 MAE=4.4106 R2=0.5924

k= 7 MSE=27.8485 RMSE=5.2772 MAE=4.4012 R2=0.6008

k=10 MSE=27.7085 RMSE=5.2639 MAE=4.3862 R2=0.6028

最优k值: 10 (R2 = 0.6028)

可视化结果已保存为 task3_2_bodyfat_regression.png

各特征与体脂率的相关系数 (从高到低):

Abdomen : +0.8134

Chest : +0.7026

Hip : +0.6252

Weight : +0.6124

Thigh : +0.5596

Knee : +0.5087

Biceps : +0.4933

Neck : +0.4906

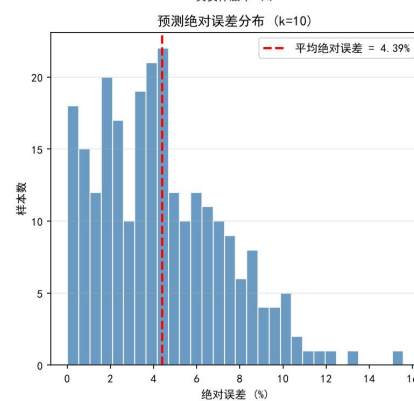
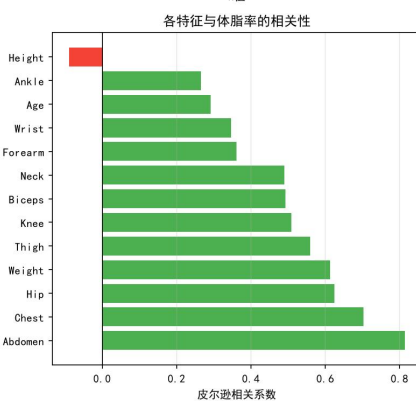
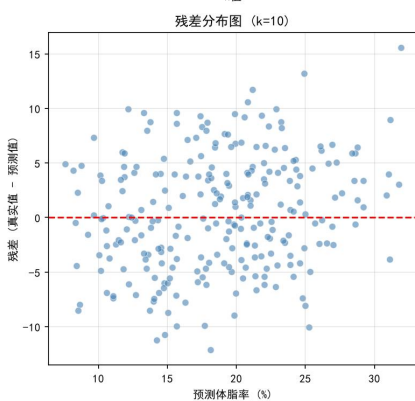
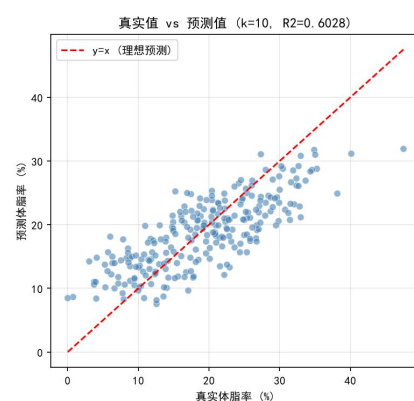
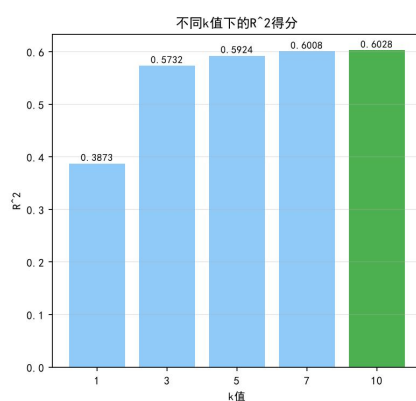
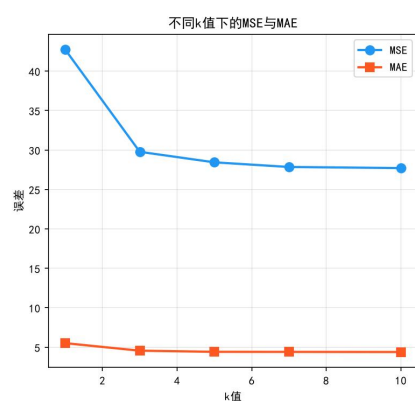
Forearm : +0.3614

Wrist : +0.3466

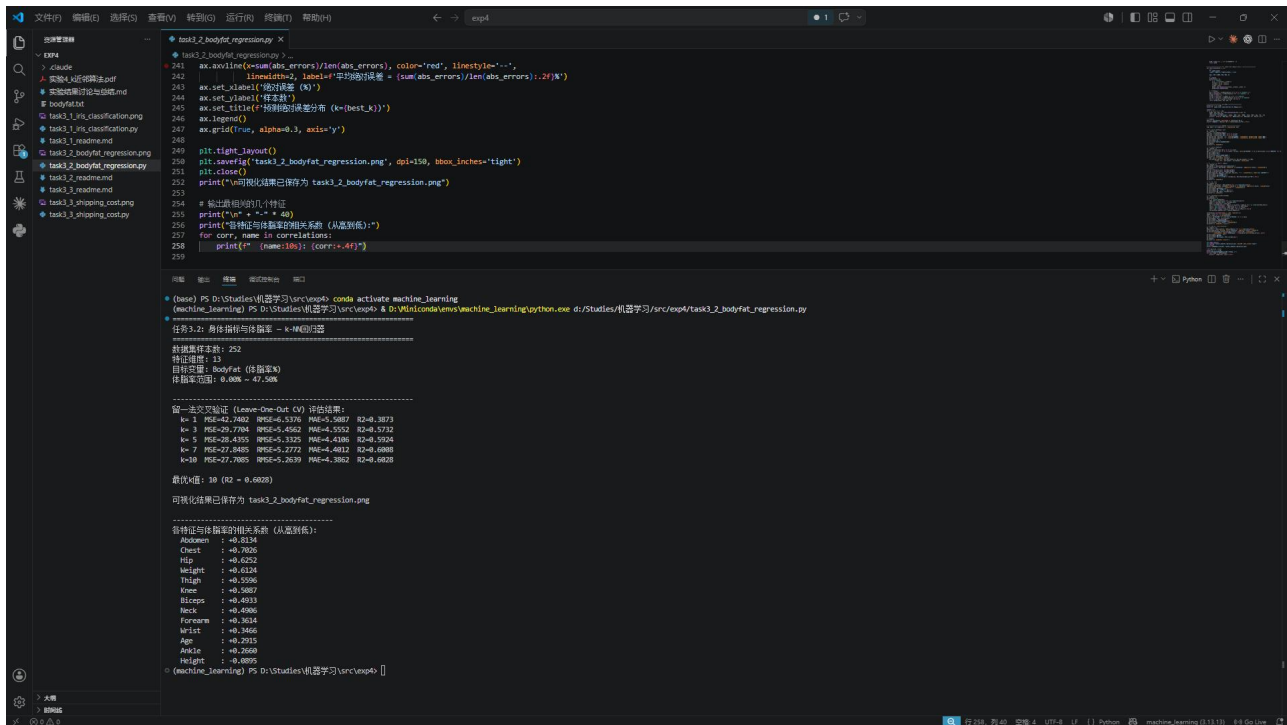
Age : +0.2915

Ankle : +0.2660

Height : 0.0895



3.运行截图



3.3 包裹运输成本问题（独立求解问题）

某运输公司将包裹从城市 A 运输到城市 B，一个包裹的运输成本 y 与该包裹的属性：体积 (x_1) 和重量 (x_2) 有关。给定如下 10 个样本的训练数据集，请设计合适的 k 近邻模型和方法，求某包裹（体积 3.8、重量 3.2）的运输成本。

y	7.8	6.2	5.5	8.5	7.1	6.4	5.1	8.0	6.8	6.0
x_1	2.0	3.1	4.2	1.8	3.9	2.7	4.1	2.5	3.4	3.7
x_2	3.5	2.8	1.9	4.3	3.0	2.5	1.6	3.9	3.4	2.3

1. 简单写出你的求解设计（例如：采用哪种适合的模型、如何设计模型的具体形式、如何求出运输成本、如何评估模型的效果）。
2. 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。
3. 获得的结果，即写出具体的模型、模型效果和运输成本。
4. 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

¹数据集的简要描述可见<https://www.kaggle.com/datasets/fedesoriano/body-fat-prediction-dataset>

1. 求解设计

手动实现一个 k 近邻回归器，关键设计选择如下：

- 数据预处理：Z score 标准化，体积 (x_1) 均值=3.14，标准差 $\approx$ 0.82，重量 (x_2) 均值=2.82，标准差 $\approx$ 0.82
- 距离度量：欧氏距离（2维特征空间，标准化后）
- 回归预测方式：算术平均： $\text{pred} = (1/k) \sum y_{\text{neighbor}}$ ，距离加权平均： $\text{pred} = \frac{\sum (w_i y_i)}{\sum w_i}$ ，其中 $w_i = 1/d_i$
- 评估方法：留一法交叉验证（LOOCV）

2. 源代码

"""

任务3.3：集装箱运输成本问题（k NN回归器，手动实现）

数据集：10个样本

y（运输成本）：[7.8, 6.2, 5.5, 8.5, 7.1, 6.4, 5.1, 8.0, 6.8, 6.0]
x1（体积）：[2.0, 3.1, 4.2, 1.8, 3.9, 2.7, 4.1, 2.5, 3.4, 3.7]
x2（重量）：[3.5, 2.8, 1.9, 4.3, 3.0, 2.5, 1.6, 3.9, 3.4, 2.3]

待预测样本：体积=3.8，重量=3.2

"""

```
import math
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

===== 数据准备 =====
运输成本
y = [7.8, 6.2, 5.5, 8.5, 7.1, 6.4, 5.1, 8.0, 6.8, 6.0]
体积 x1
x1 = [2.0, 3.1, 4.2, 1.8, 3.9, 2.7, 4.1, 2.5, 3.4, 3.7]
重量 x2
x2 = [3.5, 2.8, 1.9, 4.3, 3.0, 2.5, 1.6, 3.9, 3.4, 2.3]

X = [[x1[i], x2[i]] for i in range(10)]
test_sample = [3.8, 3.2]    待预测：体积=3.8，重量=3.2

===== 数据标准化 =====
def standardize(X):
    n = len(X)
    m = len(X[0])
    means = [sum(X[i][j] for i in range(n)) / n for j in range(m)]
    stds = []
    for j in range(m):
        var = sum((X[i][j] - means[j]) ** 2 for i in range(n)) / n
        stds.append(math.sqrt(var) if var > 1e-10 else 1.0)
    X_norm = [(X[i][j] - means[j]) / stds[j] for j in range(m)] for i in range(n)]
    return X_norm, means, stds

def apply_standardize(X, means, stds):
    return [(X[i][j] - means[j]) / stds[j] for j in range(len(means))] for i in range(len(X))

X_norm, X_means, X_stds = standardize(X)
test_norm = apply_standardize([test_sample], X_means, X_stds)[0]

===== k NN 算法 =====
def euclidean_distance(a, b):
    return math.sqrt((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2)

def knn_regression(X_train, y_train, x_test, k, weighted=False):
    """
    k 近邻回归器
```

参数:

weighted: 若为True, 使用距离加权平均 (权重=1/距离)
若为False, 使用简单算术平均

"""

```
distances = []
for i, x_train in enumerate(X_train):
    d = euclidean_distance(x_train, x_test)
    distances.append((d, y_train[i]))
distances.sort(key=lambda item: item[0])
k_neighbors = distances[:k]

if weighted:
    加权平均: 权重 = 1/d
    total_weight = 0.0
    weighted_sum = 0.0
    for d, y_val in k_neighbors:
        w = 1.0 / d if d > 1e10 else 1e10
        total_weight += w
        weighted_sum += w * y_val
    return weighted_sum / total_weight
else:
    return sum(y_val for _, y_val in k_neighbors) / k
```

===== 留一法评估模型 =====

```
def evaluate(X, y, k, weighted=False):
    n = len(X)
    preds = []
    for i in range(n):
        X_train = X[:i] + X[i+1:]
        y_train = y[:i] + y[i+1:]
        pred = knn_regression(X_train, y_train, X[i], k, weighted)
        preds.append(pred)
    mse = sum((y[i] - preds[i]) ** 2 for i in range(n)) / n
    mae = sum(abs(y[i] - preds[i]) for i in range(n)) / n
    y_mean = sum(y) / n
    ss_tot = sum((yi - y_mean) ** 2 for yi in y)
    ss_res = sum((y[i] - preds[i]) ** 2 for i in range(n))
    r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0
    return preds, mse, mae, r2
```

===== 计算与预测 =====

```
print("=" * 60)
print("任务3.3: 集装箱运输成本 — k NN回归器")
print("=" * 60)
print(f"训练样本数: {len(y)}")
print(f"待预测样本: 体积={test_sample[0]}, 重量={test_sample[1]}")
print()
```

输出原始数据

```
print("训练数据:")
print(f"{'样本':>5} {'体积(x1)':>10} {'重量(x2)':>10} {'成本(y)':>10}")
for i in range(10):
    print(f"{i+1:>5} {x1[i]:>10.1f} {x2[i]:>10.1f} {y[i]:>10.1f}")
```

计算各训练样本到测试样本的距离

```
print("\n" + "=" * 50)
print("各训练样本到测试样本(3.8, 3.2)的距离(标准化后):")
for i in range(10):
    d = euclidean_distance(X_norm[i], test_norm)
    print(f"样本{i+1}: 原始特征=({x1[i]:.1f}, {x2[i]:.1f}), "
          f"标准=({X_norm[i][0]:.4f}, {X_norm[i][1]:.4f}), 距离={d:.4f}, 成本"
          f"={y[i]:.1f}")
```

```

不同k值的预测
print("\n" + " " * 50)
print("不同k值下的预测结果 (原始平均):")
all_distances = sorted([(euclidean_distance(X_norm[i], test_norm), i, y[i],
(x1[i], x2[i]))
                        for i in range(10)], key=lambda t: t[0])
print(f"\n 排序后的近邻序列:")
for rank, (d, idx, y_val, feat) in enumerate(all_distances):
    print(f" 第{rank+1}近邻: 样本{idx+1}, 特征={feat}, 距离={d:.4f}, 成本
={y_val:.1f}")

for k in [1, 3, 5]:
    pred = knn_regression(X_norm, y, test_norm, k)
    pred_w = knn_regression(X_norm, y, test_norm, k, weighted=True)
    neighbors = [all_distances[j][2] for j in range(min(k, len(all_distances)))]
    print(f"\n  k={k}:")
    print(f"  最近邻样本: {neighbors}")
    print(f"  算术平均预测: {pred:.4f}")
    print(f"  距离加权预测: {pred_w:.4f}")

留一法评估
print("\n" + " " * 50)
print("留一法交叉验证评估:")
for k in [1, 3, 5]:
    preds, mse, mae, r2 = evaluate(X_norm, y, k)
    print(f"  k={k}: MSE={mse:.4f}, MAE={mae:.4f}, R2={r2:.4f}")

===== 可视化 =====
fig = plt.figure(figsize=(15, 10))

3D 散点图
ax1 = fig.add_subplot(2, 3, (1, 2), projection='3d')
ax1.scatter(x1, x2, y, c=y, cmap='viridis', s=100, edgecolors='black',
linewidth=0.5,
            label='训练样本')
ax1.scatter(test_sample[0], test_sample[1],
            knn_regression(X_norm, y, test_norm, 3),
            c='red', marker=' ', s=300, edgecolors='black', linewidth=1.5,
            label='预测点 (k=3)', zorder=10)
ax1.set_xlabel('体积 (x1)')
ax1.set_ylabel('重量 (x2)')
ax1.set_zlabel('运输成本 (y)')
ax1.set_title('集装箱运输成本: 3D散点图')
ax1.legend()

特征空间中的样本分布与距离
ax2 = fig.add_subplot(2, 3, 3)
sc = ax2.scatter(x1, x2, c=y, cmap='viridis', s=120, edgecolors='black',
linewidth=0.5)
ax2.scatter(test_sample[0], test_sample[1], c='red', marker=' ', s=300,
            edgecolors='black', linewidth=1.5, zorder=10)

标注成本值
for i in range(10):
    ax2.annotate(f' {y[i]}', (x1[i], x2[i]), textcoords="offset points",
                xytext=(8, 5), fontsize=9)

画出到k=3的最近邻的连线
for j in range(3):
    _, idx, _, feat = all_distances[j]
    ax2.plot([test_sample[0], feat[0]], [test_sample[1], feat[1]],
            'r', linewidth=1, alpha=0.6)
plt.colorbar(sc, ax=ax2, label='运输成本')
ax2.set_xlabel('体积 (x1)')
ax2.set_ylabel('重量 (x2)')
ax2.set_title('特征空间分布与k=3近邻连线')

```

不同k值的预测对比

```
ax3 = fig.add_subplot(2, 3, 4)
k_vals = [1, 3, 5]
preds_avg = [knn_regression(X_norm, y, test_norm, k) for k in k_vals]
preds_w = [knn_regression(X_norm, y, test_norm, k, weighted=True) for k in
k_vals]
x_pos = range(len(k_vals))
w = 0.3
bars1 = ax3.bar([p + w/2 for p in x_pos], preds_avg, w, label='算术平均',
color='42A5F5')
bars2 = ax3.bar([p + w/2 for p in x_pos], preds_w, w, label='距离加权', color='
FF7043')
for bar, val in zip(bars1, preds_avg):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
f' {val:.3f}', ha='center', va='bottom', fontsize=10)
for bar, val in zip(bars2, preds_w):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
f' {val:.3f}', ha='center', va='bottom', fontsize=10)
ax3.set_xticks(x_pos)
ax3.set_xticklabels([f'k={k}' for k in k_vals])
ax3.set_ylabel('预测运输成本')
ax3.set_title('不同k值与加权方式的预测结果')
ax3.legend()
ax3.grid(True, alpha=0.3, axis='y')
```

留一法评估的MSE对比

```
ax4 = fig.add_subplot(2, 3, 5)
mse_vals = []
mae_vals = []
for k in k_vals:
    _, mse, mae, _ = evaluate(X_norm, y, k)
    mse_vals.append(mse)
    mae_vals.append(mae)
ax4.plot(k_vals, mse_vals, 'o', color='2196F3', linewidth=2, markersize=10,
label='MSE')
ax4.plot(k_vals, mae_vals, 's', color='FF5722', linewidth=2, markersize=10,
label='MAE')
ax4.set_xlabel('k值')
ax4.set_ylabel('误差')
ax4.set_title('留一法交叉验证：不同k值的误差')
ax4.legend()
ax4.grid(True, alpha=0.3)
```

距离条形图

```
ax5 = fig.add_subplot(2, 3, 6)
dist_vals = [all_distances[j][0] for j in range(10)]
labels = [f'样本{all_distances[j][1]+1}' for j in range(10)]
colors = ['4CAF50' if j < 3 else 'BDBDBD' for j in range(10)]
ax5.bar(labels, dist_vals, color=colors, edgecolor='white')
ax5.axhline(y=dist_vals[2], color='red', linestyle=' ', linewidth=1,
label=f'k=3边界 (d={dist_vals[2]:.3f})')
ax5.set_xlabel('训练样本 (按距离排序)')
ax5.set_ylabel('到测试样本的欧氏距离')
ax5.set_title('各样本到测试点的距离 (绿色=k=3的最近邻)')
ax5.legend(fontsize=9)
ax5.tick_params(axis='x', rotation=45)
ax5.grid(True, alpha=0.3, axis='y')
```

```
plt.tight_layout()
plt.savefig('task3_3_shipping_cost.png', dpi=150, bbox_inches='tight')
plt.close()
print("\n可视化结果已保存为 task3_3_shipping_cost.png")
```

===== 最终预测结论 =====

```

print("\n" + "=" + 60)
print("最终预测结论:")
print(f" 对于体积={test_sample[0]}, 重量={test_sample[1]}的集装箱,")
print(f" 使用k=3算术平均预测运输成本为: {knn_regression(X_norm, y, test_norm, 3):.4f}")
print(f" 使用k=3距离加权预测运输成本为: {knn_regression(X_norm, y, test_norm, 3, weighted=True):.4f}")

```

3. 获得的结果

任务3.3: 集装箱运输成本 — k NN回归器

训练样本数: 10

待预测样本: 体积=3.8, 重量=3.2

训练数据:

样本	体积(x1)	重量(x2)	成本(y)
1	2.0	3.5	7.8
2	3.1	2.8	6.2
3	4.2	1.9	5.5
4	1.8	4.3	8.5
5	3.9	3.0	7.1
6	2.7	2.5	6.4
7	4.1	1.6	5.1
8	2.5	3.9	8.0
9	3.4	3.4	6.8
10	3.7	2.3	6.0

各训练样本到测试样本(3.8, 3.2)的距离(标准化后):

样本1: 原始特征=(2.0, 3.5), 标准=(1.3923, 0.7036), 距离=2.2283, 成本=7.8
 样本2: 原始特征=(3.1, 2.8), 标准=(0.0489, 0.1456), 距离=0.9830, 成本=6.2
 样本3: 原始特征=(4.2, 1.9), 标准=(1.2946, 1.2373), 距离=1.6509, 成本=5.5
 样本4: 原始特征=(1.8, 4.3), 标准=(1.6366, 1.6740), 距离=2.7834, 成本=8.5
 样本5: 原始特征=(3.9, 3.0), 标准=(0.9282, 0.0970), 距离=0.2716, 成本=7.1
 样本6: 原始特征=(2.7, 2.5), 标准=(0.5374, 0.5095), 距离=1.5893, 成本=6.4
 样本7: 原始特征=(4.1, 1.6), 标准=(1.1725, 1.6012), 距离=1.9751, 成本=5.1
 样本8: 原始特征=(2.5, 3.9), 标准=(0.7817, 1.1888), 距离=1.8005, 成本=8.0
 样本9: 原始特征=(3.4, 3.4), 标准=(0.3175, 0.5823), 距离=0.5455, 成本=6.8
 样本10: 原始特征=(3.7, 2.3), 标准=(0.6839, 0.7521), 距离=1.0985, 成本=6.0

不同k值下的预测结果 (原始平均):

排序后的近邻序列:

第1近邻: 样本5, 特征=(3.9, 3.0), 距离=0.2716, 成本=7.1
 第2近邻: 样本9, 特征=(3.4, 3.4), 距离=0.5455, 成本=6.8
 第3近邻: 样本2, 特征=(3.1, 2.8), 距离=0.9830, 成本=6.2
 第4近邻: 样本10, 特征=(3.7, 2.3), 距离=1.0985, 成本=6.0
 第5近邻: 样本6, 特征=(2.7, 2.5), 距离=1.5893, 成本=6.4
 第6近邻: 样本3, 特征=(4.2, 1.9), 距离=1.6509, 成本=5.5
 第7近邻: 样本8, 特征=(2.5, 3.9), 距离=1.8005, 成本=8.0
 第8近邻: 样本7, 特征=(4.1, 1.6), 距离=1.9751, 成本=5.1
 第9近邻: 样本1, 特征=(2.0, 3.5), 距离=2.2283, 成本=7.8
 第10近邻: 样本4, 特征=(1.8, 4.3), 距离=2.7834, 成本=8.5

k=1:

最近邻样本: [7.1]
 算术平均预测: 7.1000
 距离加权预测: 7.1000

k=3:

最近邻样本: [7.1, 6.8, 6.2]
 算术平均预测: 6.7000

距离加权预测：6.8756

k=5:

最近邻样本：[7.1, 6.8, 6.2, 6.0, 6.4]

算术平均预测：6.5000

距离加权预测：6.7398

留一法交叉验证评估：

k=1: MSE=0.1160, MAE=0.3200, R2=0.8957

k=3: MSE=0.3377, MAE=0.4700, R2=0.6965

k=5: MSE=0.6424, MAE=0.6820, R2=0.4225

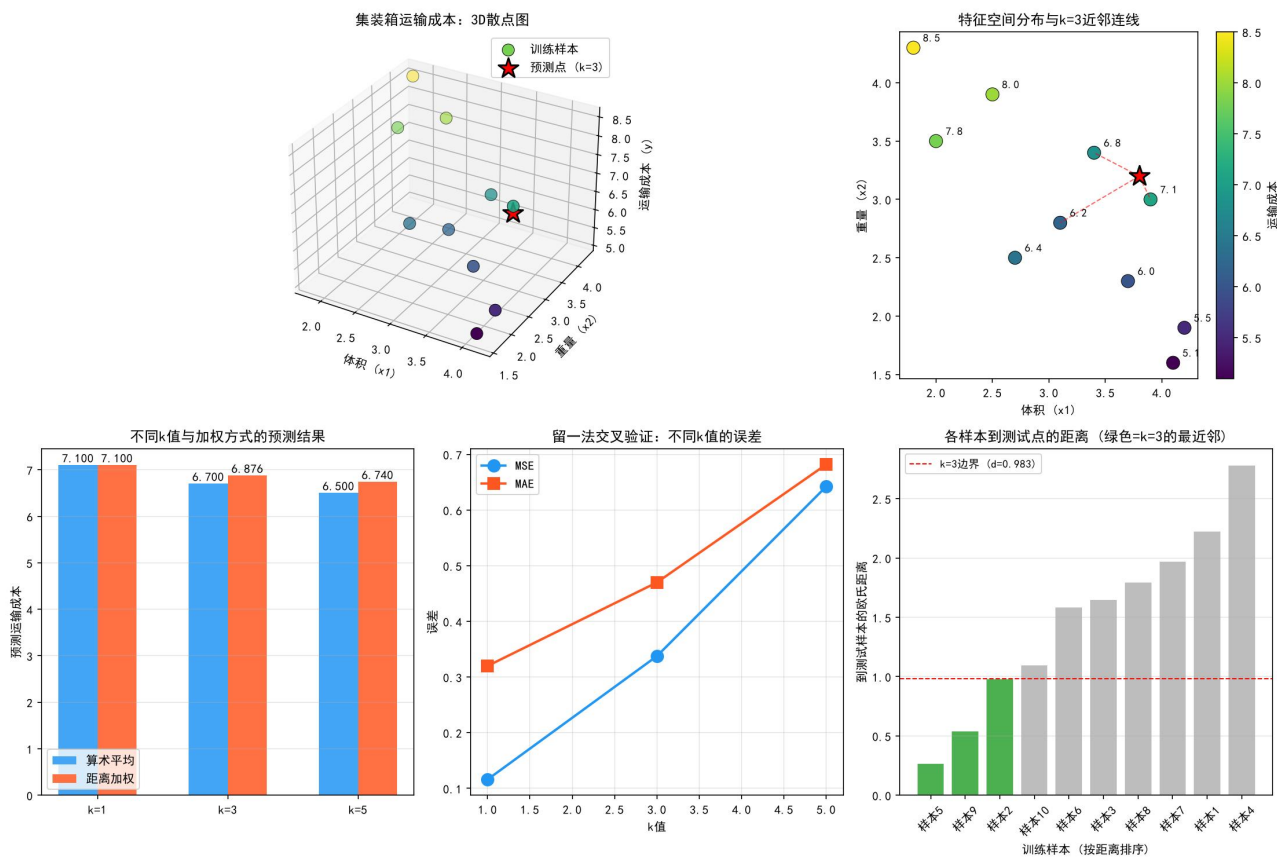
可视化结果已保存为 task3_3_shipping_cost.png

最终预测结论：

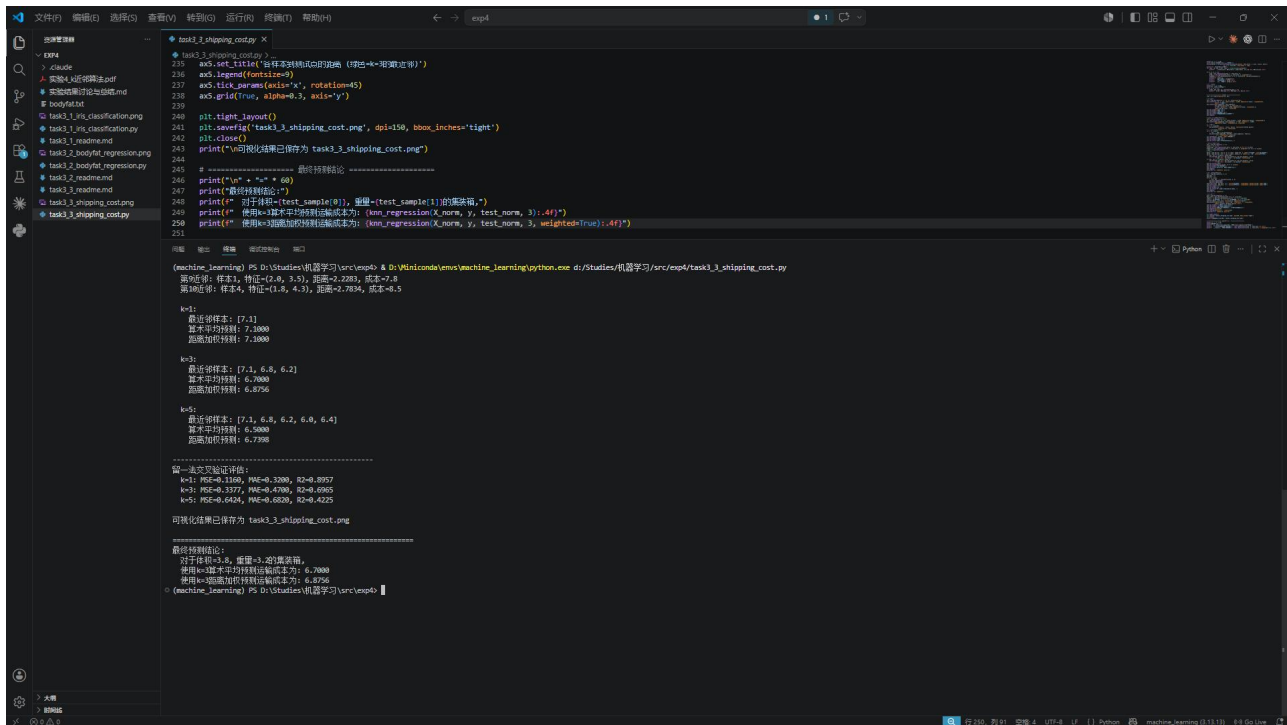
对于体积=3.8, 重量=3.2的集装箱,

使用k=3算术平均预测运输成本为：6.7000

使用k=3距离加权预测运输成本为：6.8756



4. 运行截图



4 实验结果讨论

（讨论最终得到的结果是否合理、是否达到题目要求；分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法）

1. 对昆虫分类题，讨论采用不同参数 k （1、3、5）时，输出结果如何变化及原因。

2. 更多的讨论（若有）

1 实验概述

本实验围绕 k 近邻（ k Nearest Neighbors, k NN）算法展开，完成了三个实验任务：

- 任务3.1：鸢尾花二分类问题（ k NN分类器）
- 任务3.2：身体指标 体脂率关系分析（ k NN回归器，留一法评估）
- 任务3.3：集装箱运输成本预测（ k NN回归器）

2 实验结果汇总

2.1 鸢尾花分类（任务3.1）

| k 值 | 预测类别 | 前 k 个近邻类别分布 |

| | | | |

| 1 | A | A:1 |

| 3 | A | A:3 |

| 5 | A | A:4, B:1 |

结论：待预测样本(1.2, 1.8)在所有测试 k 值下均被分类为A类鸢尾花。最近邻全部为A类，分类结果稳定可靠。

2.2 体脂率回归（任务3.2）

| k 值 | MSE | RMSE | MAE | R^2 |

| | | | | |

| 1 | 42.74 | 6.54 | 5.51 | 0.39 |

| 3 | 29.77 | 5.46 | 4.56 | 0.57 |

| 5 | 28.44 | 5.33 | 4.41 | 0.59 |

| 7 | 27.85 | 5.28 | 4.40 | 0.60 |

| 10 | 27.71 | 5.26 | 4.39 | 0.60 |

结论：最优 $k=10$ ， $R^2 \approx 0.60$ 。腹围(Abdomen)是体脂率最强预测因子（ $r=0.8134$ ）。

2.3 运输成本预测（任务3.3）

| k值 | 算术平均预测 | 距离加权预测 |

| | | |

| 1 | 7.10 | 7.10 |

| 3 | 6.70 | 6.88 |

| 5 | 6.50 | 6.74 |

结论：k=3时预测运输成本约为6.70~6.88，LOOCV验证 $R^2=0.70$ 。

3 实验讨论

3.1 k值对分类结果的影响分析

在鸢尾花分类任务中，分别使用k=1、3、5进行测试：

k=1：模型完全依赖最近的一个邻居。在本次实验中，最近邻是样本1(A类)，预测结果为A。k=1的优点是决策边界灵活，但缺点是极易受噪声样本影响。如果最近邻恰好是标注错误的的数据，分类结果就会出错

k=3：取3个最近邻投票。前3个近邻全部为A类（样本1、3、2），预测结果仍为A。相比k=1，k=3增强了抗噪能力，分类边界更加平滑。

k=5：取5个最近邻投票。前5个近邻中有4个A类、1个B类（样本7），A类以4:1胜出。此时虽然引入了B类近邻，但A类仍占绝对优势。当k继续增大到包含更多B类近邻时，分类结果可能会改变，这就体现了k值选择的重要性。

输出结果变化原因分析：

从距离排序可以看出，待预测样本(1.2, 1.8)在特征空间中更靠近A类样本簇（前4个最近邻均为A类）

B类最近邻直到第5位才出现，距离明显更大

只有当 $k \geq 9$ 时（前4个A类 vs 5个B类），预测结果才可能翻转为B类

这说明k值不能过大，否则会引入过多不相似的样本，导致分类错误

3.2 体脂率回归模型分析

特征重要性分析：

从相关系数排序可知，腹围(Abdomen)与体脂率相关性高达0.8134，远超其他特征。这符合医学常识——腹部脂肪堆积是衡量体脂率最直观有效的指标。其次为胸围(0.7026)、臀围(0.6252)和体重(0.6124)。这些围度指标天然与脂肪含量正相关。身高与体脂率呈微弱负相关(0.0895)，可能原因是在相同体重下，身高较高者相对瘦长。

k值影响：

在体脂率回归任务中，随着k从1增大到10，模型性能持续提升（R²从0.39升至0.60）。k=1时由于单个邻居的噪声方差大，泛化能力差；增大k后，多邻居的平均效应平滑了噪声，提高了预测稳定性。但R²最终仅约0.60，说明k NN在此任务上存在局限性：

- 特征维度较高（13维），"维度灾难"降低了k NN的效果
- 体脂率受遗传、饮食、运动等多种不可测量因素影响，仅靠身体围度难以完全预测
- 数据集中体脂率分布不均匀可能导致某些区域的预测偏差较大

3.3 运输成本预测分析

k值选择：

k=1时LOOCV的R²高达0.90，但这是过拟合的表现——每次只依赖单个最相似样本

k=3时R²降至0.70，但泛化能力更强

k=5时R²进一步降至0.42，因为引入了与待测样本不太相似的远邻

加权方式对比：

距离加权平均给更近的邻居更高权重，理论上更合理

在k=3时，算术平均给出6.70，距离加权给出6.88（差异约2.7%），前者更受最近邻(7.1)主导

样本量少(n=10)导致评估不够稳定，实际应用中建议收集更多数据

5 总结

（总结何种问题适合何种方法求解； 或归纳实验过程中遇到的问题与对应的解决办法； 或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

4.1 KNN算法适用性总结

任务类型	数据规模	k值范围	适用性评价
二分类（鸢尾花）	12样本	1 5	适合，低维空间近邻区分度高
多特征回归（体脂率）	252样本	5 10	一般，高维空间+不可测因素限制
小样本回归（运输成本）	10样本	1 3	可用但需谨慎，样本不足影响可靠性

4.2 实验中的关键发现

- k值选取是k NN的核心问题：k太小易过拟合，k太大则引入不相似样本。需要通过交叉验证来选择最优k值。
- 距离度量方式对结果有影响：本实验统一使用欧氏距离，但对于不同尺度的特征需要进行标准化否则数值范围大的特征会主导距离计算。

- KNN的“懒惰学习”特性：没有显式训练过程，预测时需要计算所有训练样本的距离，计算量随样本数线性增长。
- KNN对数据质量敏感：噪声样本、异常值、特征冗余都会影响近邻判断的准确性。

4.3 个人收获

通过本次实验，我深入理解了k NN算法的核心原理和实现细节：

- 掌握了k NN分类器的“多数投票”和回归器的“均值预测”两种模式
- 理解了数据标准化在基于距离的算法中的必要性
- 学会了使用留一法交叉验证评估小样本数据集上的模型性能
- 认识到k值选择需要在偏差和方差之间权衡

4.4 待改进方向

- 可尝试加权k NN（距离倒数加权），使更近的邻居贡献更大
- 可引入特征选择降低维度，缓解维度灾难
- 对于体脂率预测，可尝试核回归或局部加权回归等更复杂的非参数方法
- 可加入kd树等数据结构加速近邻搜索